PA consistency machines: comparison and search below 196 states

Research report

13 November 2026

Abstract

The requested target is a binary, single-tape, blank-start Turing machine with fewer than 196 working states whose nonhalting is equivalent to Con(PA), with the equivalence available in PA. **This investigation does not establish that target.** It supplies a reproducible comparison of the cited designs, a new 196-state rearrangement of the PA196 implementation, a reduction from 199 to 198 states within the supplied diagonal-vacuity branch, a new command-level refutation search that halts at the last inference, and additional bounded searches and inductive analyses.

The nine principal U208 output files represent two machine graphs; the expanded accepted helper family contains twenty pairwise nonisomorphic 208-state graphs. The requested James Chang PA209 is distinct from the Joint U209 ancestor of U208. The published PA196 proof explicitly exports $\mathrm{PA} \vdash \mathrm{NH} \to \mathrm{Con(PA)}$; its external nonhalting theorem must not be confused with an exported PA proof of the converse. The accompanying files distinguish literal tables, executable symbolic certificates, ordinary mathematical arguments, and outstanding formalization work. No unsuccessful finite search below is presented as a lower bound.

1 Result, scope, and conventions

There is no verified machine with at most 195 working states in this archive. The smallest new complete table supplied here has 196 working states, the same count as the supplied PA196. The second complete table has 198 states and improves the previously supplied 199-state diagonal-vacuity implementation. These are useful implementation results, but neither meets the numerical goal.

All machine counts use the same convention: one tape extending infinitely in both directions; alphabet $\{0, 1\}$; all cells initially zero; one designated initial working state; and transitions that write one symbol and move one cell left or right. A distinguished halt control is excluded from the working-state count. No larger alphabet, preloaded program tape, stationary move, or uncounted startup controller is used. A row (q, b, w, D, q') reads b, writes w, moves in direction D, and enters q'.

The dense new tables start at state 0. The 196-state table has halt label 196; the 198-state table has halt label 198. Native tables preserve sparse historical labels, including initial control 74 and halt control 254. The archive contains explicit bijections between native and dense labels.

Machine or source	States	Rows	Source sites	Status here
Requested James Chang PA209	209	418	3,948	Supplied
U208, classes A and B	208	416	3,909	Supplied
Published PA196	196	392	3,812	Baseline
New rotated PA196	196	392	3,812	Verified layout
Supplied diagonal-vacuity branch	199	398	3,700	Reference
New diagonal-vacuity layout	198	396	3,700	Verified layout
New halt-at-MP source	—	—	3,788	Source only

Table 1: "Verified layout" means the executable reconstruction, symbolic primitive contracts, and exact blank startup described below. It does not mean a new exported proof-assistant theorem of the PA biconditional.

1.1 Pinned inputs

The source snapshot is fixed by three commits:

- PA196: `jamesyc/BusyBeaver` at `407cdc356f6b586635e71ac92be394255f4bfea4`.
- Requested PA209: the same repository at `69d98241a721ead20b4f0b17df9062950b40fbfe`.
- Incoming research and ConciseIPRA: `VladimirReshetnikov/ProveIt` at `dc523e7b06ffb414093c90aa035500ca283e5f75`.

The PA196 table has SHA-256

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

The requested PA209 table has SHA-256

`0610d978675320f0b4b1ef34619dcd3daa5912632d7037e200dfcc70954fea51`.

The compact comparison manifest records original archive member names, hashes, state maps, and the selected twenty U208 representatives. It is not necessary to download the large original archives to rerun the comparison. The licenses and attribution accompanying the reused material are retained.

2 What the 208-state collection actually contains

2.1 Two principal graphs, twenty accepted representatives

Different filenames and state labels do not necessarily identify different machines. To compare rooted deterministic binary transition graphs, the checker assigns canonical names by breadth-first traversal from the stated initial control, visiting the read-0 edge before the read-1 edge. It fixes read symbols, write symbols, and movement directions. The terminal control is represented separately. Equality is checked on the complete canonical arrays, not merely on their cryptographic hashes.

Every working state in each selected table is reachable in this finite transition graph. That observation concerns the graph with both read choices available; it is not a claim that every working state is reached on the actual blank-tape execution.

There are also eighteen accepted same-count helper variants: nine helper constructions applied separately to A and B. Each replaces one control with an added helper, keeping 208 working states. Together with A and B they give **twenty** pairwise nonisomorphic tables. Their local equivalence

Principal output in the incoming archive	Graph class
U208_ConPA: principal	A
U208_ConPA_Conditional_Context: principal / alternative	A / B
U208_ConPA_Entry_Context: principal / alternative	A / B
U208_ConPA_History: principal / compatibility	A / B
u208_research: principal / plain	B / A

Table 2: The nine named principal outputs have only two rooted, action-preserving graph isomorphism classes. Exact state bijections and byte reconstruction checks are supplied.

arguments belong to the supplied research [3]; the fresh audit here establishes the literal comparison and provenance rather than re-proving every older helper theorem.

The nine helper recipes are identified by their supplied labels:

`127_124_8_109`	`127_124_8_125`	`127_124_8_59`
`178_168_154_176`	`178_168_154_225`	`178_168_159_176`
`178_168_159_225`	`190_160_86_193`	`301_88_122_500`

Exploratory or explicitly rejected tables elsewhere in the archives are not promoted to accepted constructions merely because their row count happens to equal 416.

2.2 A and B have the same register program

The two principal graphs implement the same ordered primitive word and the same ordinal control-flow graph. Their 3,909 primitive sites contain 1,558 increments, 1,175 decrements, 80 forward transfers, 1,095 backward transfers, and one halt. Their source addresses differ at 720 positions; the maximum address lengths are 42 and 41. Each decrement has two branch contracts, so the corresponding complete primitive contract count is $3909 + 1175 = 5084$.

This is a source-layout difference, rather than two independent logical axiomatizations whose savings can simply be added. The supplied archive contains source maps for both versions, and the comparison checker reconstructs and compares the entire ordinal graph.

2.3 The Joint U209 is not the requested PA209

The final U208 fold acts on the Joint U209 lineage [3], whose program has 3,909 sites. James Chang's requested 209-state predecessor has 3,948 sites. Equal state counts do not identify those parents. The common ancestry of the broader compiler family does not make their individual labels or contextual invariants interchangeable.

In the Joint parent's native labels, the distinguishing incoming rows are

$$(163, 0, 0, L, 211), \qquad (163, 1, 1, R, 211),$$

and controls 172 and 211 have the same read-1 action, namely $(1, R, 184)$. The reduction uses entry history and an existing helper to replace the read-0 behavior while folding the two controls. The original report gives positive blocks of lengths 1, 3, and 7. For its stated initial control 0 and a common arbitrary initial tape, it states the finite-time relation

$$T_{208} = T_{209} + 2N_{211,0} + 6N_{172,0}.$$

This is not a theorem for arbitrary initial control states. In one read-0 entry case, the invariant needs three preceding transitions; the later failure of a two-history overapproximation does not contradict that argument. Blank-start halting preservation is the consequence needed for the present problem.

The PA196 table has no state whose read-0 and read-1 rows respectively write 0/move left and write 1/move right to the same working target. Thus the exact mixed-entry motif above is absent. State renaming cannot transplant it. A new contextual proof, a changed layout, or a different implementation would be needed to obtain another saving.

3 Why PA196 saves thirteen states over its PA209 predecessor

Let

$$p(a, b) = \frac{(a + b)(a + b + 1)}{2} + b.$$

The PA196 constructors are

$$I(a, b) = 4p(a, b), \quad M(a, b) = 4p(a, b) + 1, \quad U(a, B) = 4p(B, a) + 2, \quad E(a, b) = 4p(a, b) + 3.$$

Code 0 is truth and code 1 is the designated contradiction $M(0, 0)$. Universal quantification reverses the two Cantor-pair arguments relative to the requested PA209 predecessor.

The axiom emitter also changes mathematically [1, 2]. It removes the old seventh equality head, strengthens the guarded gate with equality vacuity, and reorders or reverses selected head antecedents. The published paper derives the required rewriting, neighborhood equality, and witness principles without retaining the old seventh head as a primitive emission. These changes reduce the full source from 3,948 to 3,812 sites. They are not merely a thirteen-state quotient of one fixed transition table.

For the gate, if V is membership vacuity and Q is the old quantified separation component, the old gate is $G = \neg(V \to Q)$. PA196 adds equality vacuity Δ and uses

$$F = \neg(\Delta \to (V \to Q)).$$

Classically F is equivalent to $\Delta \wedge G$. This finite propositional observation alone is not the completeness proof: the freshness side conditions, quantifier coding, and derivations of the replaced equality principles still matter.

The literal 196-state machine [1] consists of 36 arithmetic-engine controls, 13 leaf-routing controls, and 147 source-reader controls. Its full source has 159 distinct internal nodes, and the axiom body is represented by 201 macro tokens and 68 concatenation nodes. Six complementary-entry folds are already included in the final table. They cannot be subtracted again when comparing with U208.

Three quantities must therefore be kept separate: the number of primitive source visits in the flattened program, the number of distinct subtrees in its shared representation, and the number of physical TM controls after allocation and contextual folding. Reducing the first or second does not automatically reduce the third.

4 A new 196-state implementation with shorter addresses

The directory `rotated196/` supplies a complete new table, its source addresses, a deterministic twelve-rotation reconstruction, dense/native conjugacy, and the executable symbolic check. It preserves the original 3,812-site primitive word and ordinal control-flow graph.

4.1 The source transformation

For a source subtree, let f and b be its escaping forward and backward depths. Increments and halt have depths $(0, 0)$, a decrement has $(1, 0)$, and forward or backward transfers of depth k have $(k, 0)$ or $(0, k)$. At an internal node the combination is

$$f(A, B) = \max\{f(A) - 1, 0, f(B)\}, \qquad b(A, B) = \max\{b(A), b(B) - 1, 0\}.$$

The directional reassociation

$$((A, B), C) \longleftrightarrow (A, (B, C))$$

preserves the ordinal source graph when $f(A) = 0$ and $b(C) = 0$. The middle subtree need not be closed. The reconstruction verifies both the condition and the complete graph after each rotation, using two independent graph resolvers.

Only pure source readers are directly rotated: a read bit is rewritten unchanged, and the head moves right. The child being rotated must have no source-reader occurrences outside the selected parent occurrences. The physical row changes implement the same local reassociation. Full prefix reconstruction is checked after every step; this is followed by the stronger complete primitive-runtime check.

Step	Parent	Child	Rotation
1	100	174	right
2	1353	1349	right
3	1348	1347	right
4	1364	1354	right
5	1354	1353	right
6	1349	1348	right
7	1349	1344	right
8	1353	1363	left
9	1360	1359	left
10	1353	1362	left
11	1363	1349	right
12	1349	1344	right

These are native labels, not working-state counts. Sparse labels above 1,000 do not add controls. The dense table has exactly 196 labels in its working domain and 392 rows.

4.2 Exact verification results

The maximum source address length drops from 38 to 36. The number of sites whose addresses obstruct the proposed shorter zero scanner falls from 19 to 14. Those remaining obstructions are why this rearrangement does not justify deleting a scanner state.

All 4,959 primitive branch contracts pass. The maximum checked symbolic execution length is 497, with 1,620,783 symbolic operations across the checks. These are symbolic operations, including normalization and register-file excursions, not a uniform bound on literal TM time for arbitrary register values. The register file and gap parameter remain symbolic. All 46 protected register-engine rows agree with the original PA196 table.

The blank startup changes. It reaches the exact clean frame after **58,294** literal transitions, compared with 58,154 for the published table. The clean frame has header gap 48, source ordinal 25, logical registers $(2, 0, 0, 0, 0, 0, 0, 0, 0)$, and the same spare-register tail. The verifier compares the entire finite tape support, state, and head position; it does not merely recognize a plausible prefix.

The new dense table's SHA-256 is

`b9873b89df4189414d0f443922ca9227bf2f76d8c81a0d0951c06cd608d6a90b`.

Exact action-preserving bisimulation still has 196 classes. Thus the new table has no immediate further quotient of that restricted kind.

4.3 Why these checks support halting preservation

At source-instruction boundaries, a frame records the current source address and the complete register file. Each successful symbolic contract supplies a positive finite machine block with the required register effect and successor address. Both branches of every decrement are included. The arithmetic-engine excursions retain their inherited contracts; the changed reader and return paths are executed explicitly.

Induction over source steps gives corresponding machine frames. The blocks have positive length and contain no unintended halt, so an infinite source execution supplies arbitrarily long nonhalting machine prefixes. Conversely, a finite machine halt cannot lie in a nonhalting primitive block and must correspond to the source halt. Together with the exact startup, this is the ordinary block-simulation argument for halting equivalence with the common source program.

The executable checker is part of the evidence, not a proof-assistant kernel [6]. Its symbolic run-length mechanism is an adaptation of the published `M6/Runtime.lean` framework and its register-file contracts. The archive does not claim that the new complete table has already been rebuilt through the entire upstream Lean development, or that the missing PA-internal consistency direction has thereby been exported.

5 A new 198-state layout of the diagonal-vacuity branch

The supplied diagonal-vacuity proposal replaces a longer gate component with the schema

$$\Gamma(B) = B \to U(B, B).$$

The numerical code of a formula is larger than every variable label occurring in it, with the truth-code convention handled separately. Consequently the variable whose label is that code is fresh in the formula, which explains the soundness of this diagonal vacuity instance. The supplied report [3] gives the separate completeness argument and keeps the membership-vacuity resources needed for its reverse rewriting.

This branch has 191 macro tokens and 3,700 primitive sites. Its supplied physical implementation uses 199 states. The new layout retains the same ordered primitive program and the same ordinal control-flow graph, but admits five compatible reader folds instead of four. The raw allocation has 203 controls, yielding

$$203 - 5 = 198.$$

The reduction is internal to this branch; it is not a reduction of the published 196-state machine to 195.

The standalone verifier reconstructs the source tree from the included pinned PA196 input and the diagonal grammar, reconstructs the raw allocation and five folds, and compares the resulting native table with the supplied new table. It checks all 46 protected engine rows directly against PA196 and verifies the dense/native bijection.

All 4,815 primitive branch contracts pass, with a maximum of 524 symbolic operations. The exact blank startup reaches the clean frame after 58,154 literal transitions.

The new dense table's SHA-256 is

`ad3cef693b0a5d3bb4e3804acd0b9092cfbefdba53cffa9783b7e582c48815e8`.

The new evidence certifies the implementation relationship with the supplied diagonal source. Its logical calibration is the inherited ordinary mathematical argument recorded in the provenance manifest; this packet does not turn that manuscript into a new exported Lean theorem.

6 A command calculus that halts at the last inference

This section gives a new command-level reduction. Its arithmetic correctness does not by itself assert a state count for a binary tape implementation. The formula constructors, guarded gate, and eight generated heads are those of PA196. Write $a :: w$ for $p(a, w)$, where p is Cantor pairing and $p(0, 0) = 0$. A store is (t, w, d, h). The underlying derivation compiler and auxiliary words are taken from `PA196/Commands/Compiler.lean`, `Words.lean`, and the selector modules at upstream commit `407cdc356f6b586635e71ac92be394255f4bfea4`. The command below and its revised MP compiler clause are manuscript constructions; they are not declarations from that Lean development.

Keep AX unchanged. Make Shift total, with update

$$(t, a :: w, d, h) \longmapsto (h, w, d, a).$$

For $\mathrm{MP}(x)$ put $b = d + x$ and use the following operation:

$$\begin{cases} \text{halt}, & I(t, b) = h \text{ and } b = 1, \\ (b, w, 0, 0), & I(t, b) = h \text{ and } b \neq 1, \\ (0, w, b, 0), & I(t, b) \neq h. \end{cases} \tag{MP†}$$

Compared with the original successful branch, the zero stack item and the copy of the consequent in the numerical cache are both removed. The halt test is applied only to a successfully derived consequent.

Soundness. Retain the PA196 invariant that t and h are derivable formula codes and that every stack item is derivable, including the infinite zero tail. The total Shift preserves this invariant. AX preserves it by its unchanged formula contracts. On a failed MP the two live formulas become truth; on a successful nonhalting MP its consequent is derivable by modus ponens and the other live formula becomes truth. A halting MP therefore yields a finite derivation of code 1. This argument constructs a derivation from the finite command trace and needs no semantic soundness theorem.

Auxiliary words. Use the same words

$$\begin{aligned} P &= \mathrm{AX}(0,0,0)\, S^8\, \mathrm{MP}(0), \\ D &= \mathrm{MP}(1)\, \mathrm{AX}(1,0,0)\, S^9\, \mathrm{MP}(0), \\ R &= \mathrm{AX}(0,0,0)\, S^{10}. \end{aligned}$$

Their contracts are

$$\begin{aligned} P(t, w, d, h) &= (0, t :: w, 0, 0), \\ D(t, w, 0, 0) &= (0, w, 0, 0), \\ R(t, a :: w, d, h) &= (t, w, 0, a). \end{aligned}$$

Every MP in P and D fails, so its new successful branch is irrelevant. In P, the compared major has universal tag 2, whereas implication codes are divisible by four. In D, the first comparison is $I(t,1)=0$, which is impossible; the final comparison is $I(U(1,0),0)=0$, also impossible. The word R has no MP at all and is now total even when the preserved top equals 1. For $i>0$, MP(i) from $(0,w,0,0)$ fails and sets $d=i$; for $i=0$ use the empty offset word. Hence the original generalization word

$$P\ \mathrm{offset}(i)\ S\ \mathrm{AX}(i,0,0)\ S^9$$

still maps $(a,w,0,0)$ to $(U(i,a),w,0,0)$. The original six head selectors, restricted-propositional selector, and gate selector use only these words and therefore retain their exact contracts.

Completeness with a shorter MP clause. Define a compiler C on finite PA196 derivations. Retain the old axiom selectors and generalization clause. For $p:A$ and $q:A\to B$, set

$$C(\mathrm{mp}(p,q)) = C(q)\,P\,C(p)\,P\,S^2\ \mathrm{MP}(b),$$

where a,b are the codes of A,B. Starting from $(t,w,0,0)$, the induction hypotheses either already give a halt or yield the sequence

$C(q);P$	$(0, I(a,b)::w, 0, 0)$
$C(p);P$	$(0, a::I(a,b)::w, 0, 0)$
S^2	$(a, w, 0, I(a,b))$
$\mathrm{MP}(b)$	halt if $b=1$; $\quad(b,w,0,0)$ otherwise.

Thus C returns the proved formula while restoring the arbitrary incoming stack and clearing both caches, or halts earlier. Every derivation of code 1 ends in MP: none of the axiom heads, guarded gates, or universal conclusions has code 1. Consequently its compiled word halts. Prepending $\mathrm{AX}(0,0,0)$ clears arbitrary incoming caches; the compiler works with whatever top and stack that command supplies. This gives a universally halting word from a refutation, exactly the property required by the cyclic enumerator.

The compiler and the trace-to-derivation construction are primitive recursive. Their correctness is proved by induction on finite derivations and finite command traces, so PA can prove their correctness after the usual coding. This is a manuscript proof, not an additional Lean artifact. The exceptional numeric identity $I(0,0)=0$ causes no problem in this argument: a successful MP with consequent 0 returns truth and continues. In particular, successful MP is not assumed to have a positive consequent.

An implementation using existing arithmetic pieces. At the entry to the old successful MP branch the live arithmetic registers satisfy $T=H=X=0$, $D=b$. The existing tree `node51` drains D into T and X, giving $T=X=b, D=0$. The existing `node19` drains X into H, giving $T=H=b, X=0$. Increment X and use the old NIP comparison on (H,X): it halts exactly when $b=1$, and otherwise clears both registers. The remaining values are $T=b, D=H=X=0$ and the unchanged stack. Shift needs only to clear T, drain H into T, and decode the next stack item into H. Relative tree-jump targets, root return placement, blank startup, and universal tape-frame contracts must all be checked for a compiled table.

The concrete reconstructed source has 3,788 instruction occurrences and 162 distinct internal tree nodes. The pinned PA196 source has 3,812 occurrences and 159 distinct internal nodes. Thus the change removes 24 instruction occurrences but adds three internal DAG nodes; it has not produced a smaller binary transition table.

The reconstruction checks all outgoing control edges of the 3,762 retained sites: old sites 11 through 3772 map to new sites 0 through 3761. The external exits are mapped by

$$3773 \mapsto 3762, \qquad 3799 \mapsto 3781, \qquad 3800 \mapsto 3782.$$

This checks the unchanged AX command, MP prelude, cyclic fetch machinery, and root stack decoder as literal control-flow fragments. Equality of operations and successors gives their usual step-for-step correspondence for every register assignment until one of the mapped boundary sites.

The new successful branch starts at site 3762. Its first loop has invariant $T = X = b - D$ and decreases D at each iteration; its second loop has invariant $H + X = b$ and decreases X. At the comparison entry the values are $T = H = b$, $D = 0$, $X = 1$. The retained comparator halts at site 3780 exactly when $b = 1$. Otherwise it clears H, X and reaches fetch site 14. Direct induction on those loops gives $9b + 9$ source transitions to fetch for $b \neq 1$, and 15 transitions to the halt instruction for $b = 1$. The new Shift at site 3782 first clears T, then transfers all of H into T, and returns through the retained stack decoder. Its two new loops terminate by descent of T and then H, respectively. No unrelated register is changed. These are arithmetic proofs about the source interpreter; the physical tape refinement remains a separate obligation.

The original literal startup is proved in `PA196/Nonhalting.lean` to reach the old fetch boundary 25 with $N = 2$ and all other registers zero. The corresponding new fetch boundary is 14, with the same source-tree path. A physical realization must bind its actual startup to this boundary; the source change alone does not establish that binding.

Reproduction evidence. The accompanying `halt_at_mp_checks.py` checks the exact integer command interpreter on 81 push cases, 25 drop cases, 243 pop cases, 27 generalization cases, 125 MP compiler clauses, and 21 positive offsets. All 522 checks pass. These finite checks supplement the universal proof above; they do not certify a binary tape table or any particular state count. The independent literal-source regression script `halt_at_mp_source_checks.py` additionally passes 240 successful branch tests, 2,286 complete MP tests, and 1,920 total Shift tests, comparing all nine registers, including the unrelated frame registers. The source snapshot and `reconstruct_halt_at_mp.py` are supplied so that the construction and the 3,762 retained control-flow sites can be checked without a Lean installation. The latter requires the pinned `PA196/Program.lean` as its input.

6.1 What its first source compilation does and does not save

The concrete source candidate is included alongside the command proof. Its primitive-site count is 3,788, a saving of 24 over PA196, and two independent ordinal graph resolvers agree. Its number of distinct internal source nodes increases from 159 to 162. No binary table is claimed for this source.

Independent finite register-interpreter checks cover the successful branch, the full MP implementation, and the total Shift, comparing all nine registers including the untouched frame. These checks supplement the universal loop and command arguments; their counts and exact input hash appear in the accompanying receipt. They do not establish a smaller physical state allocation or replace the required binary frame and startup checks.

This example shows why shorter proof words and shorter primitive programs do not settle the busy-beaver optimization problem. The semantic simplification removes the zero stack push and the consequent cache, and places the halt test precisely at a contradiction-producing inference. The first shared-tree implementation still costs more distinct nodes.

Two further cache-flexible source variants. The derivation compiler can allow the numerical cache D to have an arbitrary value after a successful MP: each compiled premise in the MP clause

is followed by PUSH, and the generalization word also begins with PUSH. Those words clear the incoming caches. Two concrete source variants were therefore tested. The first tests $b = 1$ with two decrements and retains $D = \max(b-2, 0)$ on a nonhalting success. The second makes the consequent $b = d + x + 1$, so a zero raw cache marks the contradictory consequent, and retains $D = b - 2$ on a nonhalting success. Each has 3,782 primitive sites but 163 distinct internal nodes, one more than the initial halt-at-MP source and four more than PA196. The complete MP fragments pass 2,286 and 2,304 bounded exact-integer tests, respectively, with all nine registers compared. These are source experiments; neither yields a smaller physical transition table, and the finite tests are not proof-assistant certificates. The supplied `cache_flexible_variants.py` reconstructs both from the pinned PA196 source and reproduces the receipts.

7 A stronger finite abstraction of all nine source registers

To look for overlooked constant branches, the new analysis tracks all nine registers jointly, first in three categories and then in four. For radix K, the abstraction is

$$\alpha_K(v) = \min\{v, K-1\},$$

where category $K-1$ denotes every value at least $K-1$. An abstract state is an ordinal source site and nine simultaneous categories.

Increment is exact at this level: it raises a non-top category by one and leaves the top category at top. A zero decrement takes the zero branch without changing the category. A positive non-top decrement lowers the category exactly. A top-category decrement has two possible outputs, $K-2$ and top. Ordinary jumps preserve all categories.

Lemma 7.1. *Every concrete source step is covered by one of these abstract steps. The least closed set containing the abstract clean seed therefore contains the abstraction of every concrete execution from that seed.*

Proof. For a top-category decrement, a concrete value equals $K-1$ or is larger; these give exactly the two listed possibilities after subtracting one. All other cases follow directly from the definition of α_K. Induction on the number of concrete steps gives the second assertion. □

The implementation computes this least fixed point and then separately checks closure under every abstract successor. Both runs start at ordinal 25 with registers $(2, 0, 0, 0, 0, 0, 0, 0, 0)$.

Domain for each register	Reachable abstract states	Audited edges
$\{0, 1, \geq 2\}$	4,455,507	5,000,898
$\{0, 1, 2, \geq 3\}$	32,947,398	35,969,718

Both computations find exactly the same unreachable source sites, $8, 3774, 3775$, and the same forced-zero decrement sites, $7, 3773$. Those cleanups were already present in the supplied research. Neither stronger joint abstraction identifies an additional forced-zero deletion.

These are inductive overapproximations, not sampled blank runs. They can contain an abstract halt even when the concrete machine does not halt. The result is neither a nonhalting proof nor a lower bound on possible source simplifications. Relational arithmetic facts outside these finite categories may permit other reductions.

8 Additional reduction searches and their limits

8.1 Previous work that is already in the supplied archives

The incoming collection already contains extensive negative experiments. For example, it reports 23,040 fixed-association head variants with no improvement on the original 68 distinct AX concatenation nodes, and 46,656 head-independent label permutations with no improvement on the original 159 distinct internal program nodes. Its same-count helper screen reports 8,061,300 trials without the desired saving. These are supplied results, not new searches of this investigation [3].

The same caution applies to earlier source simplifications. Removing known dead or constant branches can lower the source-site count from 3,812 to 3,807 while increasing the distinct-node count to 162 and giving a best reported 197-state allocation. Negation-head peepholes remove many more occurrences but still give 197 states in the supplied compiler. The distributed-gate branch already gives a 198-state table. The new 198-state table in this report belongs instead to the diagonal-vacuity branch. None of these counts combines additively with the contextual U208 saving.

8.2 Fresh association and reader-rotation searches

Each of three fresh AX reassociation searches used a 240-second wall-clock budget. For the original 201-token word and zero-run bound six, the run recorded 91,792 sampling attempts and a best screening cost of 62. With bound five, it recorded 99,064 attempts and a best cost of 73. For the diagonal 191-token word and bound six, 92,720 attempts found 69 distinct new readers with five potential folds, giving screening cost 64. That last witness was subsequently reconstructed and separately verified as the delivered 198-state machine.

The screening cost subtracts a maximum matching of possible context folds; it does not itself prove that all those folds can coexist. The zero-run calculation includes the temporary trailing zero introduced by the increment-one router. The attempt counter includes repeated samples and selected nodes with no admissible rotation; it does not count distinct machines. Fixed seeds and the full receipts are supplied, but repeating a wall-clock-bounded run on another machine or Python version can change its attempt total and the best result reached before the cutoff. These are heuristic search observations, not exhaustive grammar optimizations or state-count lower bounds.

A separate direct physical reader search produced the twelve-rotation 196-state layout. It expanded 412 queued layouts and accepted 11,866 prefix-decoder checks. A subsequent continuation expanded 573 layouts and accepted 16,470 prefix checks without improving the fourteen remaining zero-scanner obstruction sites. These are queue expansion counts; repeated paths can reach the same physical layout, and a prefix pass is not itself a universal primitive certificate.

A third rotation search explicitly computed action-preserving state partitions for 12,966 candidate tables after prefix checks. It found no nontrivial state quotient and no source-node count below 159 in that family. It expanded 487 queued layouts. The search is bounded by its time and queue strategy; these numbers do not assert exhaustive coverage of all tree associations or all equivalent transition tables.

8.3 Additional reader and engine searches

The new local searches did not produce a machine below 196 states. The portable packet `engine_search/` contains the search programs, pinned inputs, recorded logs, and negative receipts. These searches exhaust specified bounded certificate templates; they are not lower bounds on the number of states needed for a consistency machine.

Whole-instruction endpoints. An independent symbolic executor represents arbitrary register contents by four register-box contracts and an arbitrary additional zero gap by an affine run length. It checks all 4,959 primitive branch contracts of the 3,812-site source. The baseline completes these checks in at most 515 symbolic updates per contract; the rotated layout needs at most 497. These counts are symbolic updates, not bounds on literal tape steps.

For each eligible control, one search redirects all incoming edges to another control and tests the affected instruction suffixes against the original whole-instruction endpoint. A second search retains the read-0 row from one control and the read-1 row from the other in both mixed orientations. The bound is 1,024 symbolic updates. Neither requires the intermediate controls or head positions to match the original run.

Template and layout	Candidates	Contract trials	Survivors
Identification, baseline	33,540	35,753	0
Identification, rotated	33,540	35,561	0
Mixed rows, baseline	29,412	79,124	0
Mixed rows, rotated	29,412	74,960	0

A further incoming-edge contraction screen rejected its restricted deletions of all 172 eligible controls. None of these results excludes changed data representations, other row designs, longer simulations, or new invariants.

Two arbitrary replacement rows. Source-derived windows cover 167 nonloop controls outside the register boxes: 152,196 windows for the baseline and 152,352 for the rotated layout. Each window has radius 12 and a separate mask of known cells; a candidate may not inspect an unknown cell. The C++ screen identifies two controls, replaces both rows of the retained control arbitrarily, and permits internal revisits to that control. Each old step must be simulated within 64 new steps inside the window, restoring the old tape and head position and reaching the projected old target control.

Each layout has 27,722 directed eligible pairs and 21,734,048 initial read-0-row trials. After filtering, the baseline tests 60,990,496 paired rows and the rotated layout tests 61,912,480. Each has six candidates passing a representative-context filter, and zero passing all contexts. The six baseline candidates also fail whole-instruction checks. Additional searches for two folds sharing one new helper, or a three-control fold using one helper, find no compatible certificate. The latter tests 82 triples per layout, with 27 first-branch survivors and zero complete survivors.

Register placement. Eleven read-1 rows select the nine logical registers through the physical skip chain. A combined search assigns distinct physical slots lazily while synthesizing two replacement rows. Every bijection can be represented within this template. The screen tests 13,861 unordered pairs, 10,867,024 first rows, and 30,514,848 paired rows. Three candidates pass the representative filter; none satisfy all stored windows. *This is not an exclusion of all register permutations:* it reuses the baseline's local context obligations, while a changed physical order can alter both entry contexts and startup. For example, exchanging the physical slots of logical D and H preserves all 4,959 source contracts, but the tested blank execution does not reach the required initial frame within one million steps. That observation alone proves no nonhalting claim.

A rejected engine-tail change. Replacing native row $198/0 = 1\,L\,40$ by $0\,L\,40$ changes the spare register appended by increment. With the inherited initializer, after exactly 76 steps the machine is in state 35, its head is at cell 4, and the only ones are at cells -3 and 1. Since $35/0 = 1\,R\,35$,

induction proves that it forever sweeps right through fresh blank cells. This altered blank-start machine therefore has an elementary proof of nonhalting in PA and is rejected as a consistency machine candidate. The complete finite prefix is included. The example shows why register-runtime contracts must be accompanied by a verified blank initializer.

The symbolic engine and extracted contexts are executable analyses, not new proof-assistant theorems. Any successful candidate would still need register-box agreement, a complete blank-start certificate, unbounded source simulation, and the arithmetic proof transport.

The searches deliberately distinguish two specifications. A local transition fold must reproduce the complete tape, head position, and projected control after a positive replacement block for each parent transition. A whole-instruction test may change intermediate head positions and paths, provided it reaches the exact required source frame. The second specification is more permissive, but neither bounded family exhausts possible smaller Turing machines.

Physical register assignment is another independent choice. The published logical-to-physical permutation is

$$(6, 2, 0, 4, 3, 7, 8, 5, 1).$$

Changing it affects selector targets, possible row sharing, and blank startup. A favorable decoder census without an initialization proof does not establish a new blank-start machine. The search receipts record the scope of the register-assignment experiments actually performed; no global optimization over every compiler architecture is claimed.

8.4 A concrete warning about transplanting a valid source edit

The supplied command research already shows that Shift can test only the old H for code 1, provided its refutation compiler is adjusted. At source level this suggests replacing a transfer of old T into scratch by a clear-T loop. Merely redirecting native row $(38, 0)$ from reader 28 to reader 122 looks plausible and preserves the source prefix structure. It nevertheless fails the full primitive contract.

Reader 122 uses its entry history. At the new location it encounters the wrong preceding bit and reaches the marker for physical register 7 instead of physical register 2. A source rotation can repair that entry parity, but then a shared return path fails elsewhere. This is a useful counterexample to the inference that a correct register-level edit plus a successful prefix scan is already a correct binary implementation. The unsuccessful table is not included among the new verified machines.

9 The PA equivalence and the relevance of ConciseIPRA

9.1 The exact distinction in the published formal statements

Let H_M be the arithmetic halting sentence for the literal table and $N_M = \neg H_M$. The upstream theorem inventory explicitly includes

$$\mathrm{PA} \vdash N_M \to \mathrm{Con}(\mathrm{PA}). \tag{1}$$

The named declaration is [1] `PA196.Arithmetic.pa_proves_nonhalting_consistency`. The development also proves standard-model nonhalting and the advertised independence statements in Lean's metatheory. The source description of its HF-model soundness argument explicitly distinguishes that external argument from a PA-internal consistency proof.

To obtain the requested biconditional, one also needs

$$\mathrm{PA} \vdash \mathrm{Con}(\mathrm{PA}) \to N_M, \qquad \text{equivalently} \qquad \mathrm{PA} \vdash H_M \to \neg\,\mathrm{Con}(\mathrm{PA}). \tag{2}$$

An external proof that every accepted derivation is true in a standard model does not by itself give (2) inside PA. This is a statement about the supplied formal theorem inventory; it is not an assertion that (2) is false or unprovable.

Later incoming manuscripts already describe ordinary syntactic routes to the reverse implication [3, 4]. They should receive credit as supplied arguments, while remaining distinct from a newly exported kernel-checked endpoint. This investigation did not rebuild the full upstream Lean project. Its new executables are independent reconstruction and simulation checks.

9.2 The appropriate internal proof is a coded proof transformation

A suitable route to (2) has three components:

1. Extract an accepted finite source-command derivation from an arbitrary coded halting run of the literal machine. This requires a uniform boundary argument, including startup, total primitive blocks, and the possibility that the coded halt lies inside a final block.

2. Annotate the command execution with finite derivations of its live formula registers and stack entries. A successful MP appends a proof node; Shift transports annotations; the AX macro supplies fixed syntactic schema templates. A halt yields a derivation of code 1.

3. Translate that derivation into a PA refutation using the binary-bit interpretation of membership and uniform PA proof templates for the logical and nonlogical schemas. The separation component requires an induction instance for the translated property, supplied syntactically for each formula code.

The maps and their correctness assertions must be available uniformly on coded objects in PA, not only separately for each externally fixed standard formula. The special alias $I(0,0) = 0$, the transposed quantifier pairing, and the distinction between a formula's code and a variable's numerical label require explicit treatment.

For example, simply swapping the two Cantor components at quantifier nodes changes the numeric code of a property while leaving its old fresh binder label unchanged. A gate whose binder is constrained to equal the property code cannot be transported by that slogan alone. A generic fresh-binder template or a verified alpha-renaming step is needed. None of the new state counts in this report is used to conceal this obligation.

9.3 What can be reused from ConciseIPRA

The native scalar ConciseIPRA core works with iterator terms and equations, with its progress rule restricted to literal equations [5]. Its small proof-checker descriptions concern iPRA. Replacing a PA refutation search by that checker would change the theory being tested; equivalence to iPRA is not a refutation translation for PA.

The relevant ideas are structural. Computation can be placed in the syntax instead of rechecked by duplicated administrative fields. Proof-carrying execution can expose a direct syntactic refutation map. Redundant caches and stack padding can be removed when an exact compiler contract replaces them. The new halt-at-MP command calculus is an example of the last approach. These principles are useful, but their success must ultimately be measured in a concrete binary transition table and its proof, rather than in Python source length alone.

A fuller induction rule acting on arbitrary formulas could be studied as a route to a PA-strength presentation. Such a theory would need its own presentation and refutation-translation proofs, followed by an actual small-machine compiler. No such completed replacement is claimed here.

10 How to reproduce the delivered results

The compact deliverable uses Python 3.10 or later for the main checks. The following commands require no network access and no upstream repository checkout:

```
python3 comparison/verify.py
python3 rotated196/verify.py
python3 diagonal198/verify.py
python3 command_reduction/halt_at_mp_checks.py
python3 command_reduction/halt_at_mp_source_checks.py
```

The table verifiers deliberately reject Python optimization flags that would disable assertion-based inherited checks. The top-level `verify_all.py` runs the ordinary verification suite and reports each component's result.

The larger abstract fixed-point computations can be reproduced with a C++17 compiler:

```
g++ -O3 -std=c++17 abstract_analysis/closure.cpp -o /tmp/pa-closure
/tmp/pa-closure abstract_analysis/cfg.txt 3 /tmp/pa-radix3.json
/tmp/pa-closure abstract_analysis/cfg.txt 4 /tmp/pa-radix4.json
```

The emitted deterministic fields can be compared with the supplied receipts. The finite-domain abstraction is documented above; the raw receipts are not substitutes for running the closure checker.

To rebuild this report, run `pdflatex report.tex` twice from the archive root. The complete transition tables are separate plain-text files so that they can be consumed directly by simulators. Their checksums, state bijections, and reconstruction recipes are included.

11 Further research questions

1. Can source association and physical register assignment be optimized jointly, including startup and the shared return paths? Optimizing only the axiom word or only the existing labels can miss interactions across their boundary.

2. Can a state identification be certified at complete instruction boundaries even when every one-transition simulation fails? A useful synthesis procedure would generate a total symbolic block certificate and its register-file invariants, rather than accept finite blank-run agreement.

3. Can the new halt-at-MP calculus be compiled with fewer distinct subtrees? Its first source implementation saves 24 primitive sites but adds three internal nodes. Sharing the successful-branch transfer and the moved guard with existing blocks is the concrete next problem.

4. Can the remaining fourteen long-zero address obstructions in the rotated layout be removed without paying for additional reader states? A valid scanner reduction must account for temporary instruction bits as well as static source addresses.

5. Is there a complete hygienic-label compilation allowing the gate to be restricted to a cheaper freshness condition, such as a binder larger than both atom labels? Cofinality of available labels alone is not a proof of quantifier-instantiation and alpha-renaming completeness.

6. Can the existing annotated-proof and literal-trace blueprints be finished as an explicit PA-internal reverse implication for the exact PA196 table, then transported to verified descendants? This would separate a reusable arithmetic theorem from each future state-reduction certificate.

References

[1] James Chang, *Nonhalting and PA-independence for a 196-state Turing machine*, source, paper, and theorem inventory, pinned PA196 repository. See especially `paper/pa196.tex`, `docs/result.md`, `PA196/Independence.lean`, and `PA196/Logic/Soundness.lean`.

[2] James Chang, the requested preceding construction, pinned PA209 repository.

[3] Vladimir Reshetnikov, ProveIt incoming research, pinned incoming directory. The machine-comparison manifest gives individual archive members and their hashes, including the U208, helper, diagonal-vacuity, and command reduction reports.

[4] ProveIt, *PA196 and the incoming U208 construction*, comparison, arithmetic-bridge notes, and previous reduction searches, pinned comparison directory.

[5] ProveIt, *ConciseIPRA*, native scalar core, proof-checker proposals, and article, pinned ConciseIPRA directory.

[6] James Chang, `M6/Runtime.lean`, symbolic run-length execution and register-file contracts, pinned runtime source.